



AWS Elastic Beanstalk

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Elastic Beanstalk



Quickly deploy and manage web apps on AWS
without worrying about the underlying infrastructure

Platform as a Service

Introduction to Elastic Beanstalk

What is Platform as a Service? (PaaS)

A platform allowing customers to develop, run, and manage applications without the complexity of building and maintaining the infrastructure typically associated with developing and launching an app



Think of Elastic Beanstalk (EB)
as the Heroku of AWS



Introduction to Elastic Beanstalk

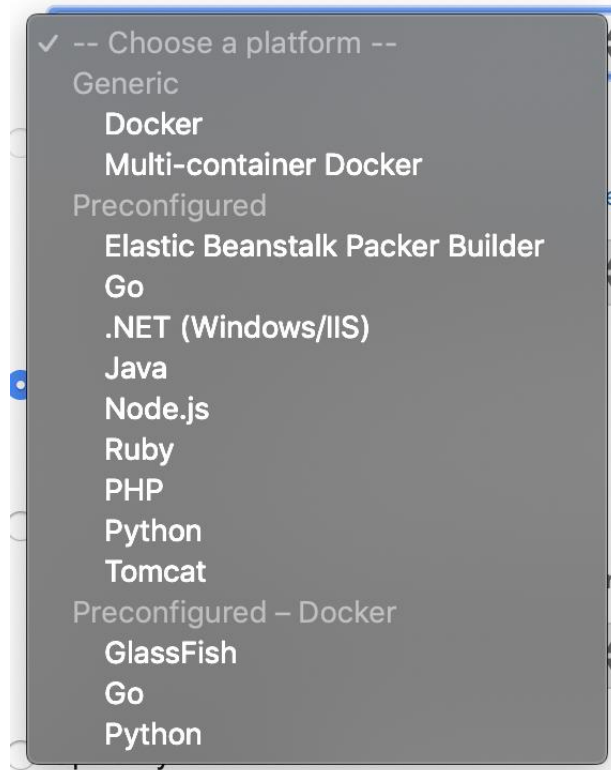
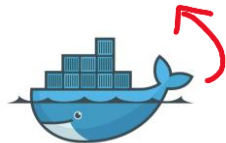
Choose a platform, upload your code and it runs with little knowledge of the infrastructure.

Not Recommended for “Production” applications

AWS is talking about enterprise, large companies.

Elastic Beanstalk is powered by a CloudFormation template setups for you:

- Elastic Load Balancer
- Autoscaling Groups
- RDS Database
- EC2 Instance preconfigured (or custom) platforms
- Monitoring (CloudWatch, SNS)
- In-Place and Blue/Green deployment methodologies
- Security (Rotates passwords)
- Can run **Dockerized** environments



EB – Supported Languages



Go



Node.js



Java



Python



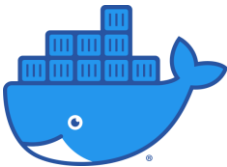
Ruby



PHP



ASP.NET



Docker

Ruby



Rails

Python



Django

PHP



Laravel

Tomcat

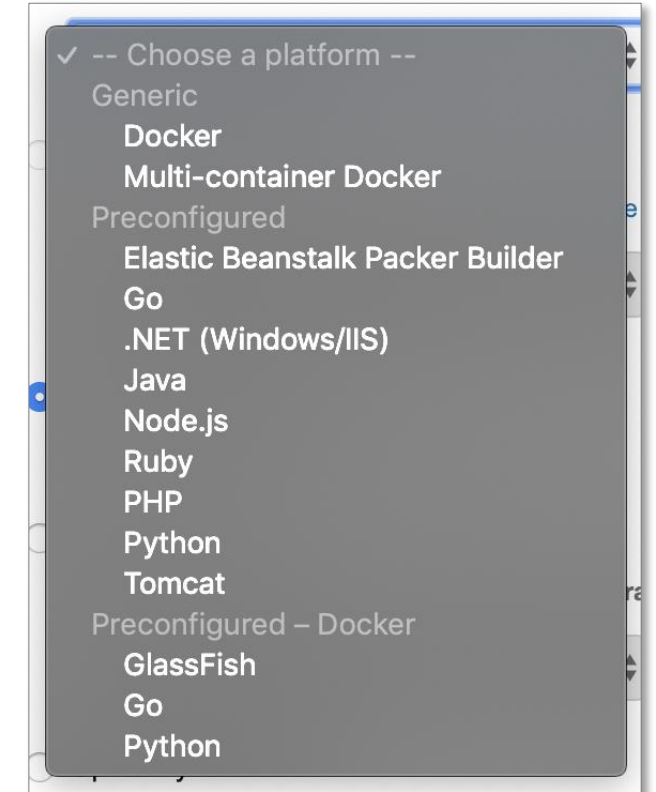


Spring

Node.js

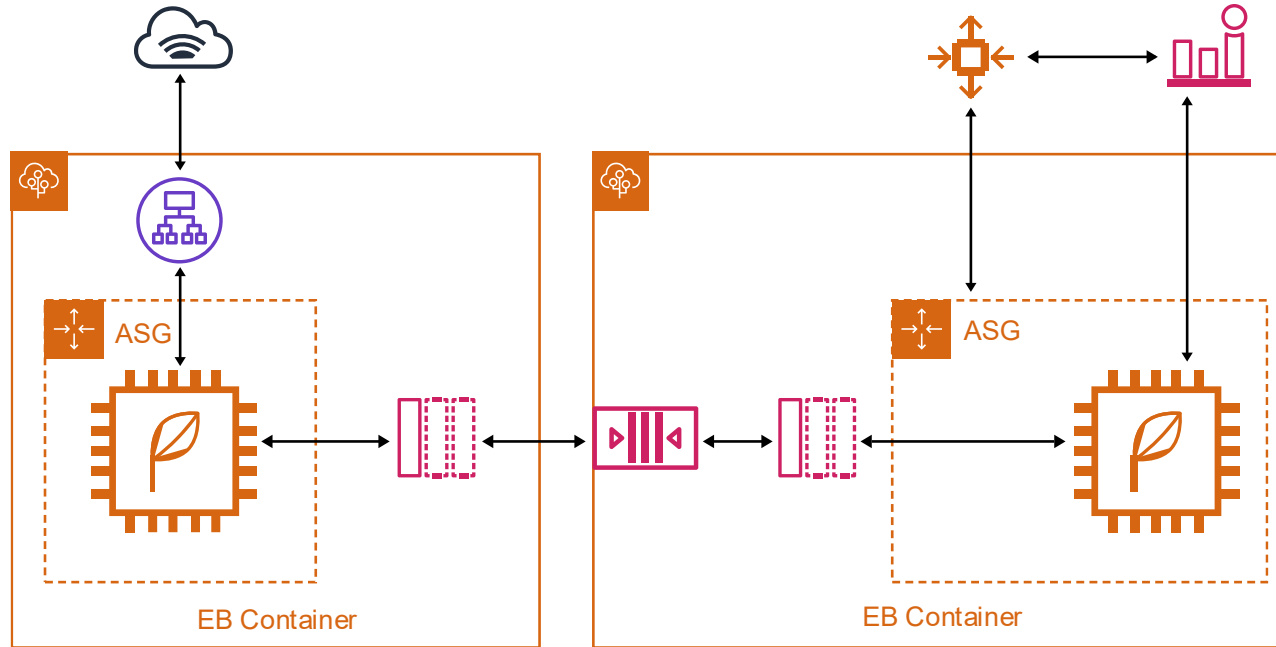


Express



EB – Web vs Worker Environment

- Creates an ASG
- **Creates an ELB**



*Web Environment

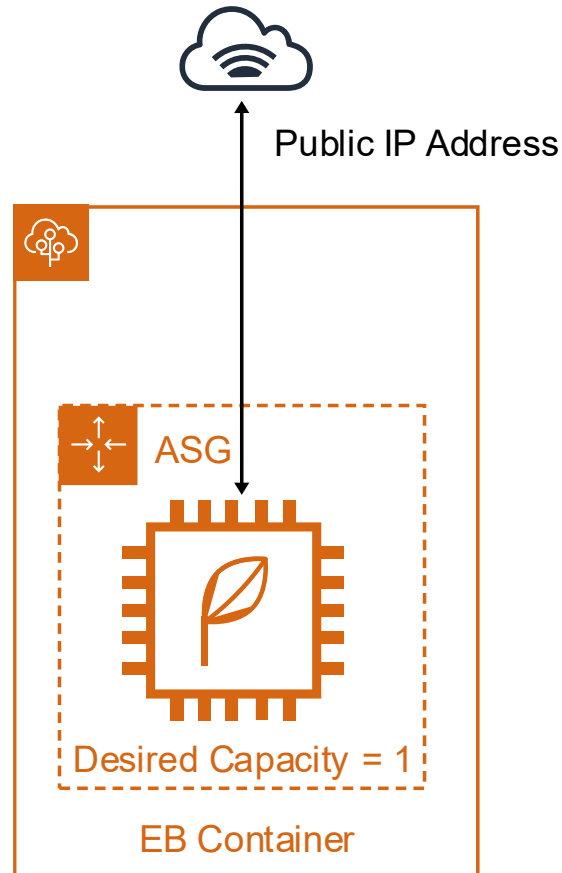
Worker Environment

comes in two variants

- Creates an ASG
- **Creates an SQS Queue**
- Installs the SQS Daemon on the EC2 instances
- Creates CloudWatch Alarm to dynamically scale instances based on health.

EB – Web Environment Types

Single-Instance Env

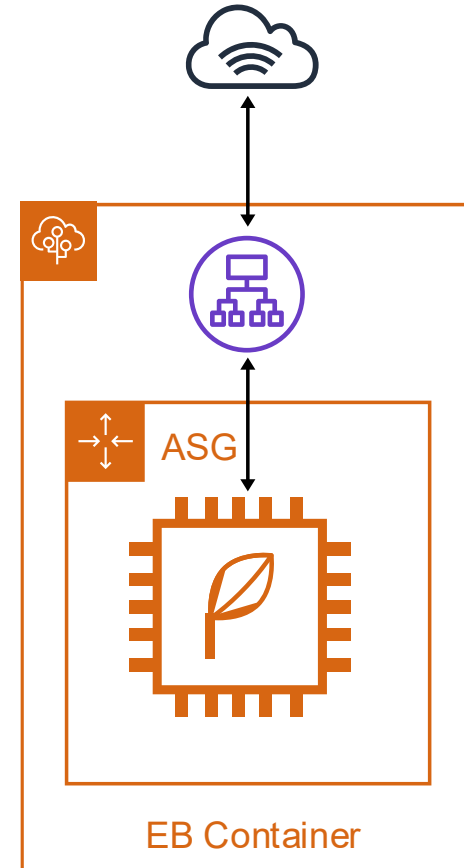


Still uses an ASG but
Desired Capacity set to 1
to ensure server is always
running.

No ELB to save on cost.

Public IP Address has to be
used to route traffic to
server

Load-Balanced Env



Uses ASG and set to scale

Uses an ELB

Designed to Scale

EB – Deployment Policies

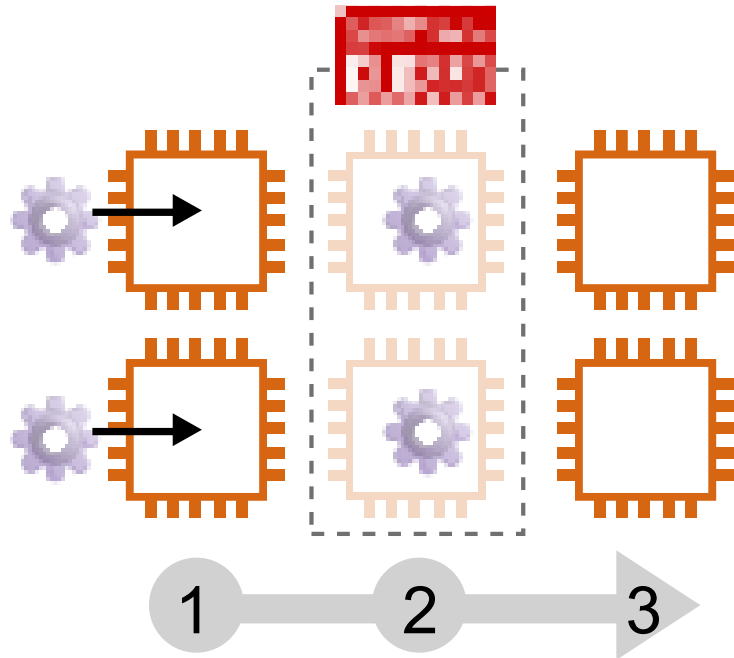
These are the Deployment Policies available with Elastic Beanstalk:

Rolling requires an ELB because they involve attaching and detaching instances in batches.

Deployment Policy	Load-Balanced Env	Single-Instance Env
All at once	👍	👍
Rolling	👍	
Rolling with additional batch	👍	👍
Immutable	👍	👍
Traffic splitting	👍 (ALB)	

Why are **In-Place** and **Blue/Green** missing on this list? We'll explain shortly.

EB – All At Once



1. Deploys the new app version to all instances at the same time.
2. Takes all instances **out of service** during the deployment process.
3. Once deployment finishes, servers become available again.

If you don't configure a deployment policy, this will be the **default** policy used



The **fastest** but also the most **dangerous** deployment method because downtime occurs for the entire application.



In Case of Failure

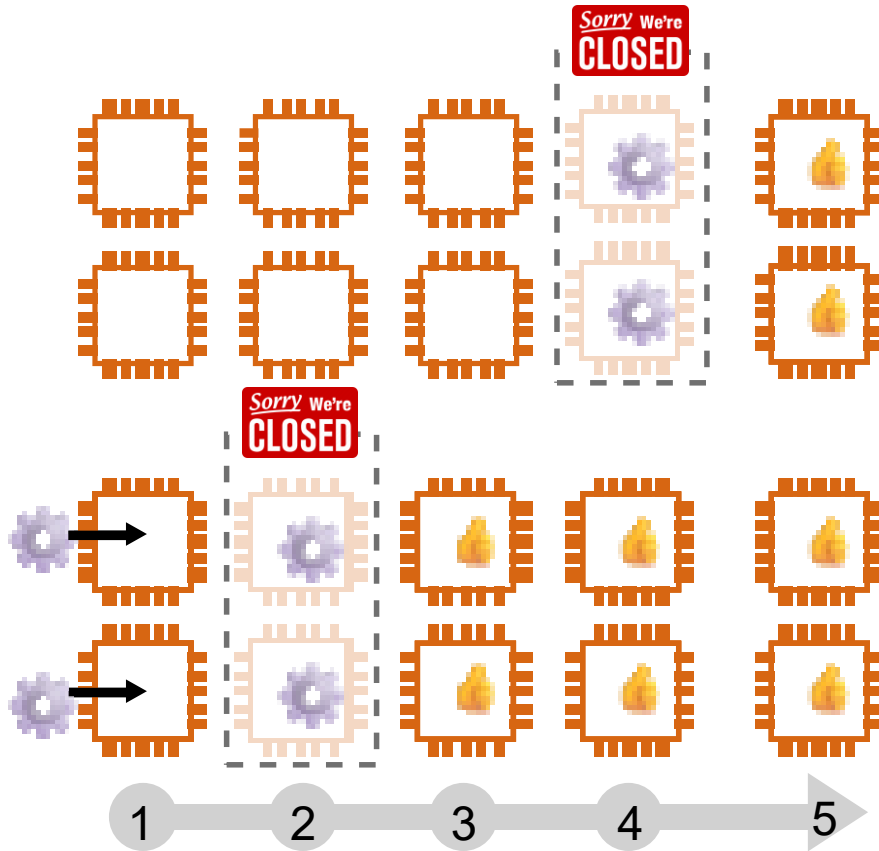
Rollback requires re-deploying the **previous version** across all instances

EB – Rolling

1. Deploys the new app version to a **batch** of instances at a time.
2. Takes batch's instances **out of service** while the deployment processes.
3. Reattaches updated instances
4. Goes onto next batch, taking them out of service
5. Reattaches those instances (rinse and repeat)

Configure in .ebextensions/deploy.config

```
option_settings:  
  aws:elasticbeanstalk:command:  
    DeploymentPolicy: Rolling  
    BatchSizeType: Percentage  
    BatchSize: 25
```



In Case of Failure

You need to perform an additional rolling update in order to roll back the changes.

EB – Rolling With Additional Batch

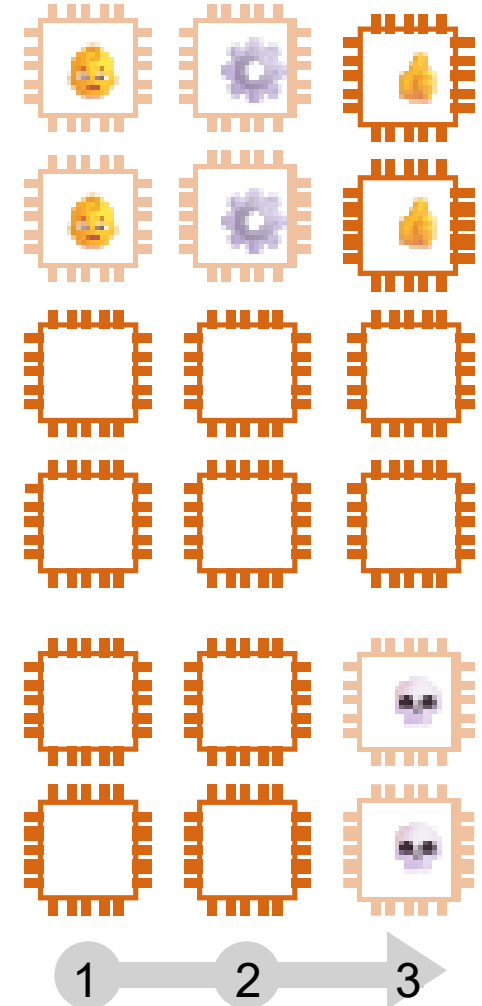
1. Launch new instance that will be used to replace a batch
2. Deploy update app version to new batch
3. Attach the new batch and terminate the existing batch

Rolling with Additional Batches ensure our capacity is never reduced.

This is important for applications where a reduction in capacity could cause availability issues for users.

Configure in `.ebextensions/deploy.config`

```
option_settings:  
  aws:elasticbeanstalk:command:  
    DeploymentPolicy: RollingWithAdditionalBatch  
    BatchSizeType: Fixed  
    BatchSize: 5
```



In Case of Failure

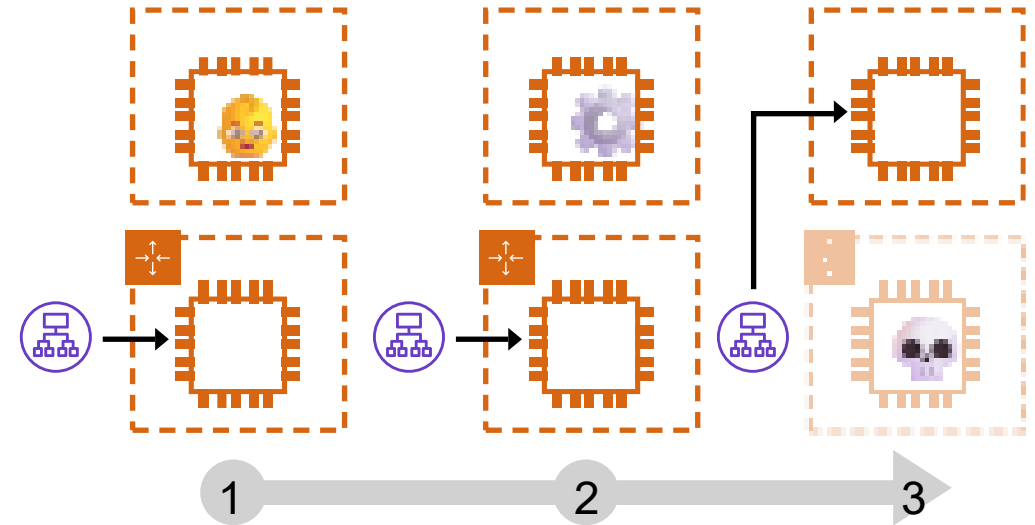
You need to perform an additional rolling update to roll back the changes.

EB – Immutable

1. Create a new ASG group with EC2 instances
2. Deploy the updated version of the app on the new EC2 instances
3. Point the ELB to the new ASG and delete the old ASG which will terminate the old EC2 instances

Configure in .ebextensions/deploy.config

```
option_settings:  
  aws:elasticbeanstalk:command:  
    DeploymentPolicy: Immutable  
    HealthCheckSuccessThreshold: Warning  
    IgnoreHealthCheck: true  
    Timeout: "600"
```



The **safest way** to deploy for critical applications



In Case of Failure

You just terminate the new instances since the existing instances still remain.

EB – Traffic Splitting

Allows you to forward a portion of your traffic to the new environment and after a period of time then move the rest.

Configure in .ebextensions/deploy.config



```
option_settings:  
  aws:elasticbeanstalk:command:  
    DeploymentPolicy: TrafficSplitting  
  aws:elasticbeanstalk:trafficsplitting:  
    NewVersionPercent: "15"  
    EvaluationTime: "10"
```



Traffic Splitting is commonly known as a Canary deployment.
The terms come from coalminers using canaries to detect if there was an issue

EB – Deployment Methods

Method	Impact of failed deployment	Deploy time	downtime	DNS change	Rollback process	Code deployed to Instances
All-at-once	Downtime		Downtime	No DNS Change	Manual	Existing
Rolling	Single batch out of service; any successful batches before failure running new application version	   †	No downtime	No DNS Change	Manual	Existing
Rolling with additional batch	Minimal if first batch fails; otherwise, similar to Rolling	   †				
Immutable	Minimal	   			Terminate New	New
Blue/green				DNS Change	Swap URL	
Traffic splitting	Percentage of client traffic routed to new version temporarily impacted	    ††		No DNS Change	Reroute traffic and terminate new instances	

† Time may vary

†† Time varies depending on evaluation time option setting

EB – In-Place vs Blue/Green Deployment

Elastic Beanstalk by default performs **in-place** updates.



In-Place and Blue/Green Deployment **are not definitive in definition** and the context can change the scope of what they mean.

This is what the exams will be referring to

In-Place could mean within
the scope of Elastic Beanstalk Env



All the **deployment policies** provided by EB could be considered In-Place since they are within the scope of a single EB environment

- All at once
- Rolling
- Rolling with additional batch
- Immutable

In-Place could mean within
the scope of the same server
(not replacing the server)

Deployment policies which do not involve the server being replaced

- All at once
- Rolling

In-Place could mean within
the scope of an uninterrupted server

Traffic is never routed away from the server (taken-of-service). Implements **Zero-downtime** deploys where Blue/Greens occurs on the server

EB can't do this. Capistrano + Ruby on Rails + Unicorn is famous case of this method of deployment

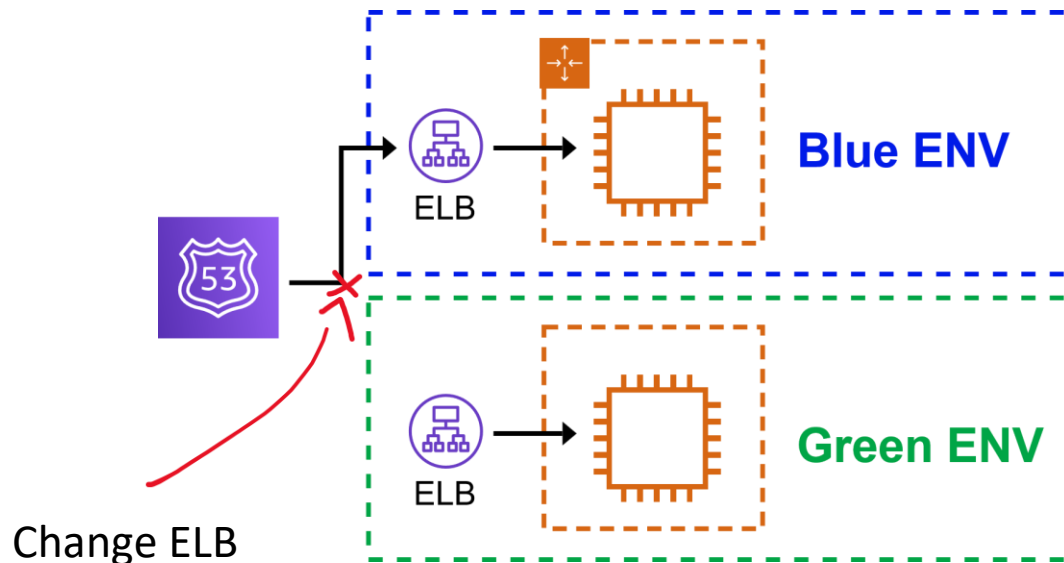
EB – In-Place vs Blue/Green Deployment

Blue / Green deployment is when you **run two identical production environments** and *shift* traffic from Blue (old) to Green (new).

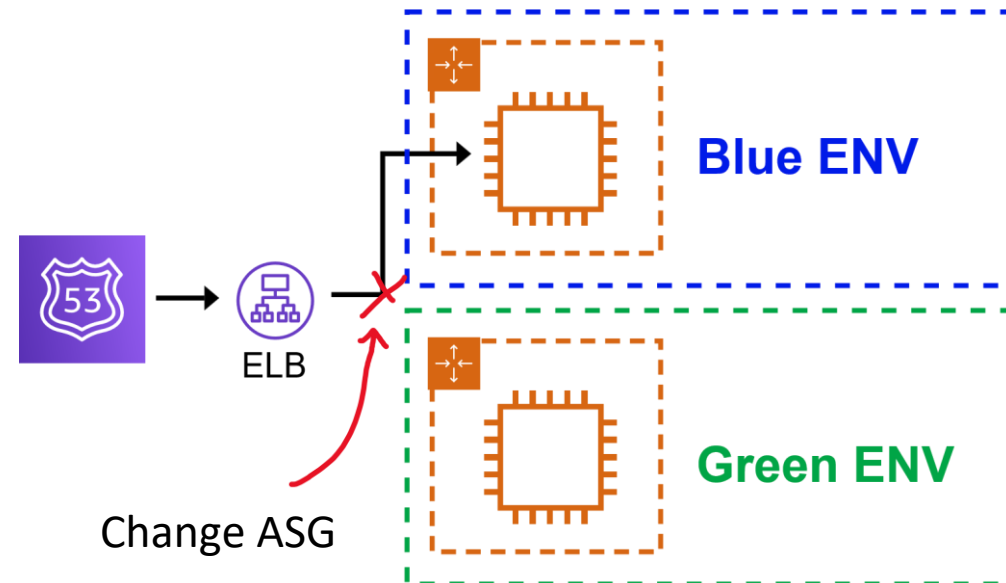
Blue could terminate after a successful health check or remain running for a defined period of time in case of rollback or Blue could require manual intervention to terminate the old environment

These Blue / Green examples below is **not what** Blue / Green is for EB, but are general examples of Blue/Green elsewhere in AWS.

DNS Blue / Green



ELB Blue / Green

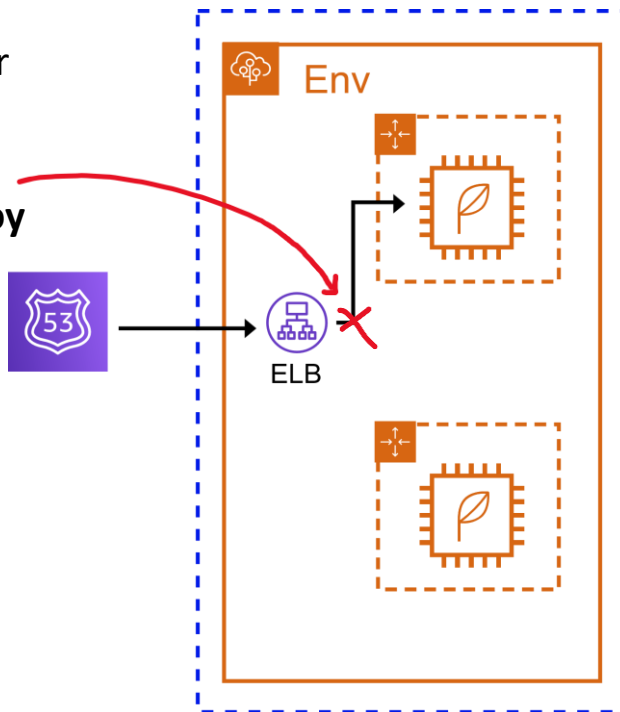


EB – In-Place vs Blue/Green Deployment

Blue / Green deployment in the **context** of Elastic Beanstalk:

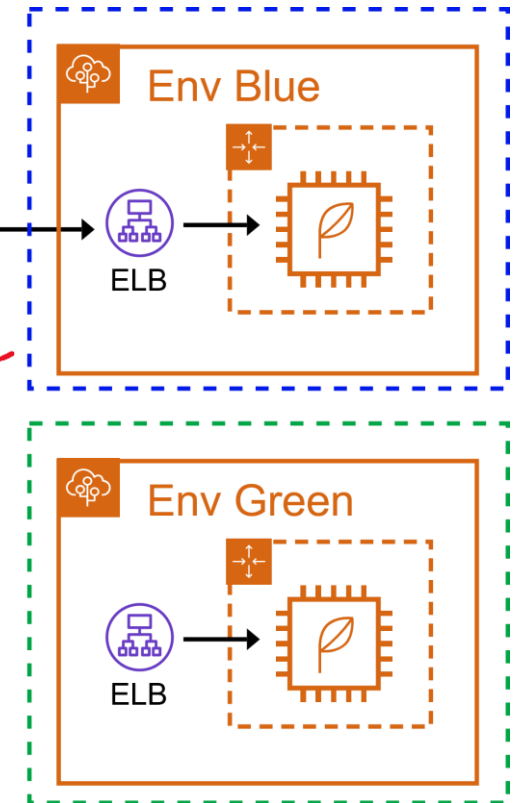
Even though swapping ASG is a Blue / Green deployment strategy, our Environment is in the EB container and **so the mechanism of our deploy is happening within** our environment, so it's considered an In-Place deployment

Immutable (In-Place)



Swap ENV URL (Blue / Green)

Elastic Beanstalk defines Blue / Green as swapping EB environments, and this occurs at the DNS level swapping out the ELBs



Blue/Green with EB requires that your database is **outside** of EB envs (because envs are terminated with loss of all their resources).

EB – Configuration Files

Elastic Beanstalk environments can be **customized** using configuration files.

.ebextensions is a hidden folder called at the root of your project which contains the config files.

.config is the extension for the config files which need to be stored in .ebextensions

Configuration files can config:

1. Option Settings
2. Linux / Windows Sever Configuration
3. Custom Resources

EB – env.yml

The **Environment manifest** is a file called **env.yml** which is stored at the root of your project.

When creating new Elastic Beanstalk environments, this file allows you to configure the defaults such as:

- The name of the environment
- Choosing the Stack Solution
- Associating the environment links
- Default Configuration of Services
- etc...

```
AWSConfigurationTemplateVersion: 1.1.0.0
EnvironmentName: exampro-prod+
SolutionStack: Ruby
EnvironmentLinks:
  "WORKERQUEUE": "worker+"
OptionSettings:
  aws:elb:loadbalancer:
    CrossZone: true
```

This file format includes support for environment groups. To use groups, specify the environment name in the manifest with a + symbol at the end.

An env.yml makes your EB environment portable.

EB – Linux Server Configuration

```
packages:  
  yum:  
    libmemcached: []  
    ruby-devrel: []
```

Packages

download and install prepackaged applications and components

```
users:  
  andrew:  
    groups:  
      - groupAdmin  
    uid: 87  
    homeDir: "/andrew"
```

Users

create Linux/UNIX users

```
groups:  
  groupsAdmin: {}  
  groupDev:  
    gid: "12"
```

Groups

create Linux/UNIX groups and assign them to group IDs

```
files:  
  "/home/ec2-user/application.yml":  
    mode: "000755"  
    owner: root  
    group: root  
    content: "Secret: 000destruct0"
```

Files

create files on the EC2 instance (inline or from URL)

EB – Linux Server Configuration

```
services:
  sysvinit:
    nginx:
      enabled: true
      ensureRunning: true
```

Services

define which services should be started or stopped when the instance is launched

```
commands:
  1_project_root:
    command: mkdir /var/www/app
  2_link:
    command: ln -s /var/www/app /app
```

Commands

execute commands on the EC2 instance before app is setup

```
container_commands:
  0_collectstatic:
    command: "django-admin.py collectstatic --noinput"
  1_syncdb:
    command: "django-admin.py syncdb --noinput"
  2_migrate:
    command: "django-admin.py migrate"
  3_customize:
    command: "scripts/customize.sh"
```

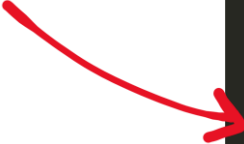
Container Commands

execute commands that affect your application source code

EB – CLI

EB CLI lets you create and manage your EB environments **from your command line**.

Example of creating a new EB env
with a **create wizard prompt**



```
eb create
Enter Environment Name
(default is eb-dev): eb-dev
Enter DNS CNAME prefix
(default is eb-dev): eb-dev
WARNING: The current directory does not contain any source code. Elastic Beanstalk is
launching the sample application instead.
Environment details for: elasticBeanstalkExa-env
  Application name: elastic-beanstalk-example
  Region: us-west-2
  Deployed Version: Sample Application
  Environment ID: e-j3pmc8tscn
  Platform: 64bit Amazon Linux 2015.03 v1.4.3 running Docker 1.6.2
  Tier: WebServer-Standard
  CNAME: eb-dev.elasticbeanstalk.com
  Updated: 2015-06-27 01:02:24.813000+00:00
Printing Status:
INFO: createEnvironment is starting.
-- Events -- (safe to Ctrl+C) Use "eb abort" to cancel the command.
```

To **install** the EB CLI you download a python script from Github and run it to install.



```
git clone https://github.com/aws/aws-elastic-beanstalk-cli-setup.git
./aws-elastic-beanstalk-cli-setup/scripts/bundled_installer
```

EB – CLI

Common Actions

- `eb init` — Initializes a new Elastic Beanstalk application, creating the necessary configuration files.
- `eb create` — Creates a new environment and deploys an application to it.
- `eb clone` — Creates a clone of an existing environment with the same configuration.
- `eb deploy` — Deploys your application to an existing environment.
- `eb logs` — Retrieves logs from your Elastic Beanstalk environment.
- `eb events`: Displays recent events for the environment.
- `eb health` — Displays health information about the environment.
- `eb terminate` — Terminates the environment and deletes all associated resources.
- `eb abort`: Cancels an environment update or deployment that is in progress.
- `eb status`: Shows the status and health of the environment.

EB – CLI

Other Actions

- `eb appversion`: Manages application versions, including listing and deleting them.
- `eb codesource`: Specifies the code source for the application, such as a local directory or a version label.
- `eb config`: Manages environment configurations, including saving, loading, and updating settings.
- `eb console`: Opens the AWS Elastic Beanstalk Management Console for your environment.
- `eb labs`: Provides access to experimental features not included in the standard EB CLI.
- `eb list`: Lists all environments associated with the application.
- `eb local`: Allows running and testing your application locally before deploying it to Elastic Beanstalk.
- `eb open`: Opens the application URL in a web browser.
- `eb platform`: Manages platforms for your environments, allowing you to create and use custom platforms.
- `eb printenv`: Displays the environment variables set for the environment.
- `eb restore`: Restores a terminated environment within a limited time frame after termination.
- `eb scale`: Changes the number of running instances in the environment.
- `eb setenv`: Sets environment variables for the environment.
- `eb ssh`: Opens an SSH connection to an instance in your environment.
- `eb swap`: Swaps the CNAMEs (URLs) between two environments.
- `eb tags`: Manages tags for resources within the environment.
- `eb upgrade`: Updates the environment to use the latest solution stack version.
- `eb use`: Sets an environment as the default for terminal session commands.

EB – Static Files

Elastic Beanstalk supports **configuring the proxy to serve static files** on most platform branches based on Amazon Linux 2 and Amazon Linux 2023

Docker environment for EB cannot perform static server proxy

.ebextensions/static-files.config

```
option_settings:  
  aws:elasticbeanstalk:environment:proxy:staticfiles:  
    /html: statichtml  
    /images: staticimages
```

On the Python and Ruby platforms, EB configures some static file folders by default. F

EB – Custom Image

When you create an EB environment, you can specify an AMI to use instead of the standard EB AMI

A custom AMI can **improve provisioning times** when instances are launched in your environment

If **you need to install a lot of software** that isn't included in the standard AMIs.



Java with Tomcat

Elastic Beanstalk supports the following Tomcat platform versions.

Platform Version and Solution Stack Name	AMI	Language	AWS X-Ray	Application Server	Proxy Server
Java 8 with Tomcat 8.5 version 3.3.2 64bit Amazon Linux 2018.03 v3.3.2 running Tomcat 8.5 Java 8	2018.03.0	Java 1.8.0_232	3.1.0	Tomcat 8.5.50	Apache 2.4.41 (default), Apache 2.2.34, Nginx 1.16.1
Java 7 with Tomcat 7 version 3.3.2 64bit Amazon Linux 2018.03 v3.3.2 running Tomcat 7 Java 7	2018.03.0	Java 1.7.0_231	3.1.0	Tomcat 7.0.94	Apache 2.4.41 (default), Apache 2.2.34, Nginx 1.16.1

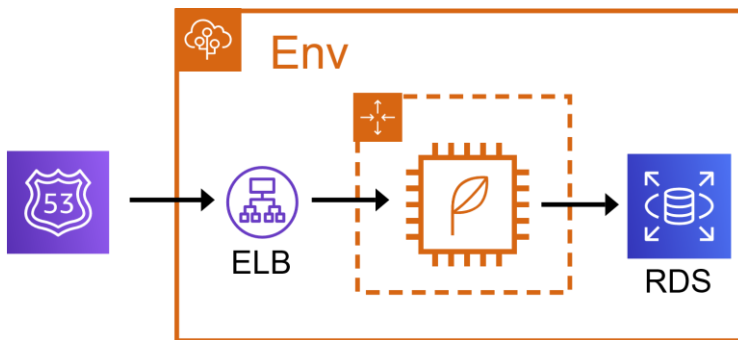
```
$ aws elasticbeanstalk describe-platform-version --  
region us-east-2 \  
    --platform-arn "arn:aws:elasticbeanstalk:us-  
east-2::platform/Tomcat 8.5 with Java 8 running on  
64bit Amazon Linux/3.1.6" \  
    --query PlatformDescription.CustomAmiList  
[  
  {  
    "VirtualizationType": "pv",  
    "ImageId": ""  
  },  
  {  
    "VirtualizationType": "hvm",  
    "ImageId": "ami-020ae06fdda6a0f66"  
  }  
]
```

EB – Configuring RDS

A database can be added **inside** or **outside** your EB Environment.

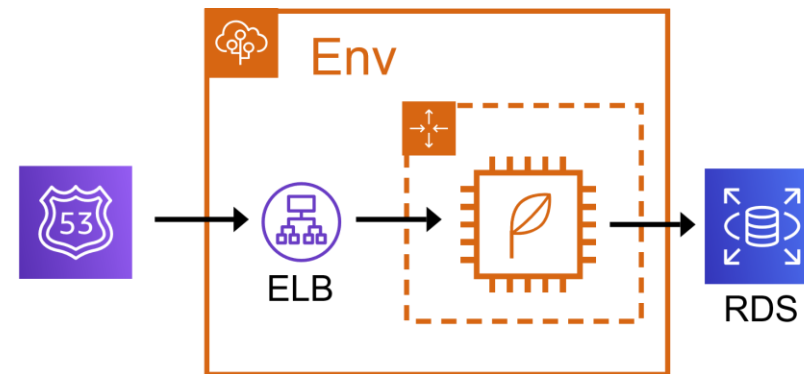
Inside EB ENV

- Intended for generally **development** envs
- You create the database within EB.
- When the EB Env is terminated the database will also be terminated.



Outside EB ENV

- Intended for **production** envs
- You create the database from RDS separate for EB
- When the EB Env is terminated, the database will remain.



EB – HTTPS Configuration

Elastic Beanstalk **allows you to use an SSH/TLS certificate** so you have HTTPS connection for your web-application

You can have HTTPS terminate at the load balancer

- You would create an ACM Certificate

You can have HTTPS terminate at the instance level

- You need to **create and update your own certificate to ACM**

```
aws iam upload-server-certificate \  
--server-certificate-name elastic-beanstalk-x509 \  
--certificate-body file://https-cert.crt \  
--private-key file://private-key.pem
```

To configure instance-level termination you need to configure each environment with your certificate.

For example for Ruby you are **overriding the nginx configuration** and including the certificate.

```
files:  
/etc/nginx/conf.d/https.conf:  
content: |  
    # HTTPS server  
  
    server {  
        listen      443;  
        server_name localhost;  
  
        ssl          on;  
        ssl_certificate /etc/pki/tls/certs/server.crt;  
        ssl_certificate_key /etc/pki/tls/certs/server.key;  
  
        ssl_session_timeout 5m;  
  
        ssl_protocols TLSv1 TLSv1.1 TLSv1.2;  
        ssl_prefer_server_ciphers on;  
  
        location / {  
            proxy_pass http://my_app;  
            proxy_set_header Host $host;  
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
            proxy_set_header X-Forwarded-Proto https;  
        }  
        # ...  
    }  
  
/etc/pki/tls/certs/server.crt:  
content: |  
    -----BEGIN CERTIFICATE-----  
    certificate file contents  
    -----END CERTIFICATE-----  
  
/etc/pki/tls/certs/server.key:  
content: |  
    -----BEGIN RSA PRIVATE KEY-----  
    private key contents # See note below.  
    -----END RSA PRIVATE KEY-----
```



Elastic Beanstalk Cheat Sheet

- **Elastic Beanstalk** handles the deployment, from capacity provisioning, load balancing, and auto-scaling to application health monitoring
- When you want to run a web application, but don't want to think about the underlying infrastructure.
- It costs nothing to use Elastic Beanstalk (only the resources it provisions eg. RDS, ELB, EC2)
- Recommended for testing or developing apps. Not recommended for production use
- You can choose from the following pre-configured platforms: Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker
- You can run containers on EB either in Single-container or Multi-container. These containers are running on ECS instead of EC2
- You can launch either a **Web Environment** or a **Worker Environment**
 - **Web Environments** come in two types **Single-Instance** or **Load Balanced**
 - **Single-Instance Env** launches a single EC2 instance, and an EIP is assigned to the EC2
 - **Load Balanced Env** launch EC2s behind an ELB managed by an ASG
 - **Worker Environments** creates an SQS queue, installs the SQS daemon on the EC2 instances, and has ASG scaling policy which will add or remove instances based on queue size.
- EB has the following **Deployment Policies**:
 - **All at once** takes all servers out-of-service, applies changes, puts servers back-in-service, Fast, has downtime
 - **Rolling** updates servers in batches, reduced capacity based on batch size
 - **Rolling with additional batch** adds new servers in batches to replace old ones, never reduces the capacity
 - **Immutable** creates the same amount of servers and switches all at once to new servers, removing old servers



Elastic Beanstalk Cheat Sheet

- Rolling deployment policies **require an ELB**, so it cannot be used with Single-Instance Web Environments.
- In-Place deployment is when a deployment occurs within the environment, all deployment policies are In-Place.
- Blue/Green is when deployment swaps environments (outside an environment). For example, you have external resources such as RDS which cannot be destroyed its suited for Blue/Green
- **.ebextensions** is a folder that contains all configuration files
- With EB, you can provide a **Custom Image** which can improve provisioning times
- If you let Elastic Beanstalk create the RDS instance, that means when you delete your environment, it will delete the database. This setup is intended for development and test environments.
- **Dockerrun.aws.json** is similar to an ECS Task Definition file and it defines multi-container configuration.